

SUPPORT DE COURS — DEV FRONT

UE03.A · HTML, CSS, Responsive, Git & première ouverture JavaScript

14 h · 4 modules de 3 h 30

HTML

CSS

Flexbox

Responsive

GitHub

Fil rouge : construire puis faire évoluer une page Web personnelle, versionnée sur GitHub.

VUE D'ENSEMBLE

Objectifs et logique du parcours

Ce que l'étudiant saura faire

- Préparer un environnement de développement simple
- Structurer une page HTML avec des balises sémantiques
- Mettre en forme avec CSS
- Construire un layout Flexbox
- Adapter une interface au mobile et au desktop
- Versionner et publier son travail sur GitHub

Fil rouge

- Choisir un thème libre : CV, médiathèque, catalogue, guide, fan page...
- Construire une page au fil des séances
- Conserver le même dépôt GitHub
- Présenter la version responsive finale
- Ajouter une petite interaction JavaScript



Écrire → versionner → partager → tester → expérimenter

Principe pédagogique — 20 à 45 minutes d'explication ou démonstration, puis une mise en pratique concrète sur le fil rouge.

1. Préparer son environnement de développement

Avant d'écrire du code, on prépare une chaîne de travail simple et reproductible.

Outils

- VS Code : éditeur principal
- Git : historique local
- GitHub : dépôt distant et partage
- Navigateur + DevTools : exécution et diagnostic
- CodePen : expérimentation rapide

Premiers réflexes

- Un dossier par projet
- Un fichier index.html à la racine
- Sauvegarder souvent
- Faire des commits courts et explicites
- Pull avant de commencer, push quand une étape est stable

Flux Git minimal (à comprendre, même si VS Code fournit une interface graphique)

```
git clone https://github.com/mon-compte/mon-projet.git
cd mon-projet
git status
git add .
git commit -m "Initialisation du projet"
git push
```

À retenir — Git versionne les fichiers. GitHub héberge le dépôt et facilite la collaboration. VS Code sait piloter Git sans quitter l'éditeur.

MODULE 1

2. Première page HTML : structure minimale

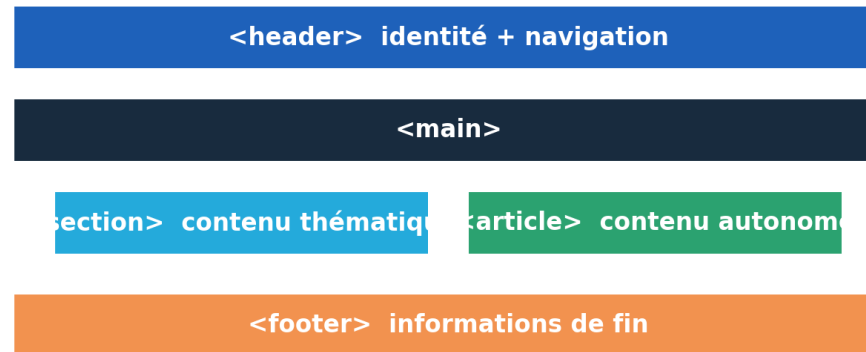
```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Mon projet</title>
</head>
<body>
  <h1>Bienvenue</h1>
  <p>Ma première page.</p>
</body>
</html>
```

- `<!doctype html>` indique au navigateur qu'il s'agit d'un document HTML moderne.
- `<html>` contient tout le document.
- `<head>` porte les métadonnées invisibles à l'écran.
- `<body>` contient ce qui sera affiché à l'utilisateur.
- `<meta name="viewport">` est indispensable pour un comportement responsive correct.

Atelier fil rouge — Créer `index.html`, modifier le titre, ajouter une phrase de présentation et pousser le premier commit sur GitHub.

MODULE 1

3. La sémantique du Web



La sémantique décrit le rôle du contenu avant sa mise en forme.

```
<header>
  <nav>...</nav>
</header>
<main>
  <section>
    <h2>Mes projets</h2>
    <article>...</article>
  </section>
</main>
<footer>...</footer>
```

Balises à privilégier

- header : en-tête
- nav : navigation

Quand utiliser div ?

- Quand aucun élément sémantique ne décrit correctement le rôle
- Pour regrouper des éléments à des fins de style ou de layout

- main : contenu principal
- section : groupe thématique
- article : contenu autonome
- footer : fin de page ou de section

- Éviter une page composée uniquement de div

MODULE 1

4. Contenu courant : texte, liens, images, listes

```
<h1>Ma médiathèque</h1>
<p>Une sélection de mes œuvres préférées.</p>
<a href="https://example.com">Voir la source</a>

<ul>
  <li>Science-fiction</li>
  <li>Roman</li>
</ul>
```

- Un seul h1 par page est une règle pédagogique simple pour démarrer.
- Les niveaux de titres h2, h3... structurent l'information.
- alt décrit l'image lorsqu'elle ne peut pas être perçue.
- href définit la destination d'un lien, src la source d'une ressource.
- Les listes ul/ol sont préférables à des suites de div lorsque l'information est réellement une liste.

Mini défi — Construire une section avec un titre, une image, une description, une liste et un lien. Puis vérifier le HTML dans DevTools.

MODULE 2

5. CSS : passer de la structure à la présentation

```
<!-- recommandé -->
<link rel="stylesheet" href="css/style.css">

/* css/style.css */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  color: #20252b;
}

h1 {
  color: #1258b7;
}
```

Trois façons d'intégrer du CSS

- Inline : `style="..."` — utile pour comprendre, rarement souhaitable
- Interne : `<style>...</style>` — pratique pour une démo
- Externe : fichier `.css` — solution retenue pour le projet

Pourquoi externe ?

- Sépare contenu et présentation
- Facilite la maintenance
- Permet de partager le même style entre plusieurs pages
- Rend le code plus lisible

MODULE 2

6. Sélecteurs CSS et CSS Diner

```
/* élément */
p { color: #333; }

/* classe */
.card { padding: 1rem; }

/* descendant */
.card h2 { margin-top: 0; }

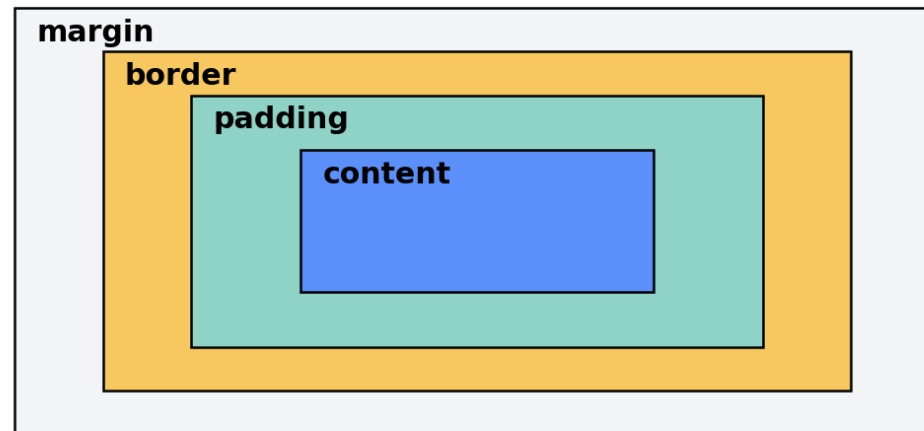
/* pseudo-classe */
a:hover { text-decoration: underline; }
```

- Le sélecteur choisit les éléments concernés.
- La propriété décrit ce que l'on modifie.
- La valeur précise le résultat souhaité.
- Une classe peut être réutilisée sur plusieurs éléments.

Atelier ludique — Utiliser CSS Diner pour travailler les sélecteurs avant de revenir sur la page du fil rouge.

MODULE 2

7. Comprendre le Box Model



```
.card {  
  width: 320px;  
  padding: 24px;  
  border: 1px solid #ddd;  
  margin: 16px;  
  box-sizing: border-box;  
}
```

- content : contenu réel de la boîte
- padding : espace intérieur
- border : bordure
- margin : espace extérieur
- box-sizing: border-box simplifie les calculs de dimensions

MODULE 2

8. Construire une identité visuelle simple

Hiérarchie visuelle

- Un titre principal clairement identifiable
- Des tailles cohérentes entre h1, h2, texte
- Une palette courte
- Des espacements réguliers
- Des composants récurrents (boutons, cartes)

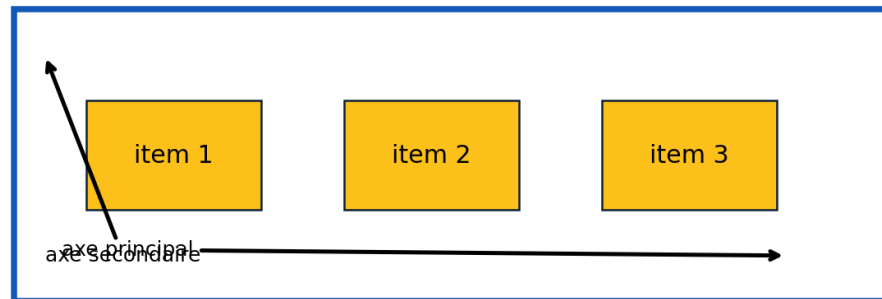
```
:root {  
  --primary: #1258b7;  
  --accent: #fcc01a;  
  --space: 1rem;  
}  
  
.button {  
  background: var(--primary);  
  color: white;  
  padding: .75rem 1rem;  
  border-radius: .5rem;  
}
```

Exemple de tokens

- --color-primary: #1258b7
- --color-accent: #fcc01a
- --space-sm: .5rem
- --space-md: 1rem
- --radius: .75rem

MODULE 3

9. Flexbox : organiser les éléments



```
.cards {  
  display: flex;  
  gap: 1rem;  
  justify-content: space-between;  
  align-items: stretch;  
  flex-wrap: wrap;  
}  
  
.card {  
  flex: 1 1 220px;  
}
```

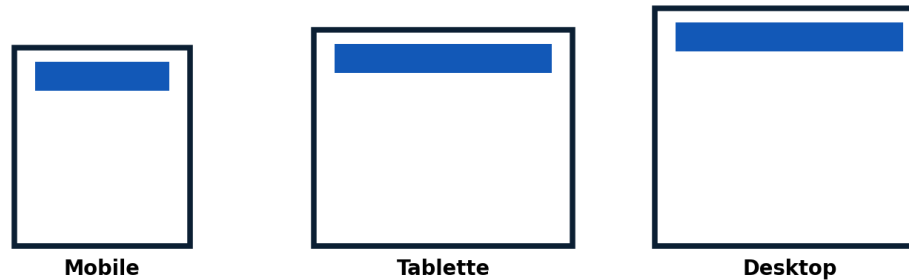
- `display: flex` active le modèle Flexbox sur les enfants directs.
- `justify-content` agit principalement sur l'axe principal.
- `align-items` agit sur l'axe secondaire.
- `gap` crée un espace régulier.
- `flex-wrap` autorise le retour à la ligne.

Atelier ludique — Flexbox Froggy pour consolider les propriétés, puis reproduction de la logique dans le projet.

MODULE 3

10. Responsive Design et Mobile First

Même contenu, mise en page adaptée à l'espace disponible



```
/* Mobile par défaut */
.cards {
  display: flex;
  flex-direction: column;
}

/* Écrans plus larges */
@media (min-width: 768px) {
  .cards {
    flex-direction: row;
    flex-wrap: wrap;
  }
}
```

Mobile First

- Styles de base pensés pour les petits écrans
- On enrichit ensuite pour davantage d'espace
- Moins de surcharges CSS
- Priorité au contenu essentiel

Tests

- DevTools responsive mode
- Largeur très étroite
- Tablette
- Desktop

- Zoom navigateur 200 %

MODULE 3

11. Déboguer avec les DevTools

- Inspecter un élément et ses règles CSS
- Activer/désactiver une propriété
- Modifier temporairement une valeur
- Comprendre quelles règles sont surchargées
- Observer la taille calculée d'une boîte
- Tester différents formats d'écran

Méthode — Quand un style ne fonctionne pas : 1) inspecter l'élément, 2) vérifier que la règle est chargée, 3) vérifier le sélecteur, 4) vérifier une éventuelle surcharge, 5) modifier temporairement dans DevTools.

MODULE 4

12. JavaScript : première interaction

```
<button id="themeButton">Changer le thème</button>
<script src="js/app.js"></script>

// js/app.js
const button = document.querySelector('#themeButton');

button.addEventListener('click', () => {
  document.body.classList.toggle('dark');
});
```

Le JavaScript change une classe ; le CSS reste responsable du rendu visuel

```
body.dark {
  background: #111827;
  color: #f9fafb;
}
```

Objectif — Comprendre que JavaScript ajoute du comportement. Il ne s'agit pas encore d'un cours complet de programmation.

MODULE 4

13. Bases de programmation en 15 minutes

```
const score = 12;           // variable

if (score >= 10) {           // condition
  console.log('Validé');
}

for (let i = 0; i < 3; i++) { // boucle
  console.log(i);
}

function saluer(nom) {       // fonction
  return `Bonjour ${nom}`;
}
```

- Variable : stocke une valeur.
- Condition : choisit une branche selon un test.
- Boucle : répète une action.
- Fonction : regroupe une logique réutilisable.

MODULE 4

14. Un vrai projet : ce qui change à grande échelle

```
project/  
├── src/  
│   ├── components/  
│   ├── services/  
│   └── pages/  
├── public/  
├── tests/  
├── package.json  
└── README.md
```

- Le code est découpé en composants et services.
- Les données peuvent venir d'API.
- Des outils construisent et optimisent l'application.
- Les tests automatisés réduisent les régressions.
- Le versionnement et la revue de code deviennent collectifs.

Message clé — Le but n'est pas de tout comprendre maintenant, mais de voir où mènent les bases étudiées.

ÉVALUATION

15. Check-list de finalisation du projet

- HTML valide et structure sémantique cohérente
- Un fichier CSS externe et organisé
- Navigation lisible
- Au moins une zone construite avec Flexbox
- Aucune rupture majeure sur mobile
- Images avec texte alternatif pertinent
- Pas de débordement horizontal
- Dépôt GitHub à jour
- README court : sujet, objectif, instructions
- Petite interaction JavaScript optionnelle mais expliquée

Présentation finale — Chaque étudiant montre sa page, explique un choix de structure, un choix CSS, une difficulté et sa résolution.

INVENTAIRE — SANS CRÉATION À CE STADE

Ressources pédagogiques à produire ensuite

- Code exemple F01 — squelette HTML minimal commenté
- Code exemple F02 — page sémantique complète header/nav/main/section/article/footer
- Code exemple F03 — galerie/cartes HTML + CSS
- Code exemple F04 — démonstration Box Model
- Code exemple F05 — Flexbox : navigation, cartes, centrage, wrap
- Code exemple F06 — page Mobile First avec deux breakpoints
- Code exemple F07 — interaction JavaScript `classList.toggle()`
- Fiche courte F-A — Git/GitHub dans VS Code : clone, commit, pull, push
- Fiche courte F-B — HTML : 25 balises essentielles
- Fiche courte F-C — CSS : sélecteurs, cascade, héritage, spécificité
- Fiche courte F-D — Box Model
- Fiche courte F-E — Flexbox en une page
- Fiche courte F-F — Responsive & Mobile First
- Fiche courte F-G — DevTools : diagnostic express
- Lexique Front — HTTP, URL, navigateur, client, serveur, HTML, CSS, DOM, sélecteur, propriété, breakpoint, responsive, commit, push, pull...
- Checklist projet final Front
- Sujet atelier fil rouge + variantes de thèmes
- Grille d'auto-évaluation étudiant
- Grille d'évaluation formateur
- Mini-quiz de fin de module 1, 2, 3 et 4

POUR ALLER PLUS LOIN

Références et documentation

- MDN Web Docs — Référence des éléments HTML : <https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Elements>
- MDN — Introduction à la mise en page CSS : https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Introduction
- MDN — Responsive design : https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
- VS Code — Source Control : <https://code.visualstudio.com/docs/sourcecontrol/overview>
- GitHub Docs — Get started with Git : <https://docs.github.com/en/get-started/using-git>
- CodePen : <https://codepen.io/>
- CSS Diner : <https://flukeout.github.io/>
- Flexbox Froggy : <https://flexboxfroggy.com/>